

Process Launcher

PCR - Parallel Command Runner

User Manual

Software version 0.2.0-rc.2

Document	PCR-UM-001
Revision	1.1
Date	9 October 2026
Author	Fation Coga
Organisation	EGO Group S.r.L
Contact	coga.fation@gmail.com
Project page	https://github.com/CogaF/ProcessLuncher
Classification	Proprietary - for users of Process Launcher

Copyright © 2025-2026 Fation Coga. All rights reserved.

Legal notices

Copyright

Copyright © 2025-2026 Fation Coga. All rights reserved. This document - its text, structure, tables and examples - is an original work of Fation Coga, author of Process Launcher and holder of its license system. It may be used by users of Process Launcher for their own use with the software. Any other reproduction, distribution, publication, translation or modification, in whole or in part, needs the written permission of Fation Coga (coga.fation@gmail.com). The licence of the source code of Process Launcher (see LICENSE and README.md) does not extend to this document.

Trademarks

Company names, product names and brand names mentioned in this document - among them Microsoft (Windows, Visual Studio, Visual C++, PowerShell, SmartScreen), GitHub - are trademarks or registered trademarks of their respective owners. They are used only to name the software Process Launcher works with (descriptive use). Process Launcher and Fation Coga are not affiliated with, sponsored or endorsed by any of these owners.

Third-party software

Process Launcher is built with the wxWidgets library, used under the wxWindows Library Licence. The commands of cmd.exe and of the Windows tools quoted in the examples are defined by Microsoft and described in its documentation; this document only states how Process Launcher uses them.

Disclaimer

This document and the example batch files are provided "as is", without warranty of any kind. A command started by Process Launcher runs with the rights of the user who started Process Launcher: check every command and batch file before using it on equipment or data that matter.

Document control

Revision	Date	Author	Description
1.0	9 October 2026	Fation Coga	First issue: commands terminated with the window, rows unlocked after a run, batch files saved in their own encoding.
1.1	9 October 2026	Fation Coga	Version 0.2.0-rc.2: projects (rows kept), time limit and Stop per row, :Exit: and {id} in the expected result, PCR_CMD_ID, Repeat with summary, CSV export, command line, screenshots.

This manual describes Process Launcher version 0.2.0-rc.2 (released 2026-10-09) on Microsoft Windows 10 and 11, together with the corrections made after that release which change what the program does (see the document control table above and the release notes).

Questions about this document, license requests and problem reports go to the author at coga.fation@gmail.com, or to <https://github.com/CogaF/ProcessLuncher/issues>.

Contents

1	Introduction	5
1.1	Purpose	5
1.2	Typographic conventions	5
1.3	Terms	5
2	Installation	5
2.1	Requirements	5
2.2	Packages	5
2.3	Files and folders	6
3	The main window	6
3.1	Command rows	6
3.2	The list of results	7
3.3	Menus	7
4	Running commands	7
4.1	How a command runs	7
4.2	Run command(s)	8
4.3	Single commands	8
4.4	Repeat	8
4.5	Time limit and Stop	8
4.6	Commands that share the result file	8
4.7	Closing the program	9

- 5 Expected result and result file 9**
 - 5.1 Forms of the expected result 9
 - 5.2 The result file 9
 - 5.3 When the result file does not exist 9
- 6 Projects 10**
- 7 Exporting the results 10**
- 8 Command line 10**
- 9 Writing batch files 11**
 - 9.1 The PASS / FAIL pattern 11
 - 9.2 Rules 12
- 10 The batch file editor 12**
 - 10.1 Encoding of the saved file 12
 - 10.2 Closing 12
- 11 License 12**
 - 11.1 Getting a license 13
- 12 Troubleshooting 13**
- 13 Appendix A - Keyboard shortcuts 14**
- 14 Appendix B - Example batch files 15**

1 Introduction

1.1 Purpose

Process Launcher (PCR, Parallel Command Runner) starts a list of commands - batch files, programs, anything `cmd /c` understands - in parallel or one after the other, looks in their output (or in a result file) for an expected text and records every run as **PASS** or **FAIL**, with pass / fail counters for each command.

Typical uses: functional and end-of-line tests driven by batch files, checks of a PC or a test bench before a session (network, services, disk space, tools installed), repeated runs of the same command to catch intermittent failures, and smoke tests after an installation.

1.2 Typographic conventions

Convention	Meaning
Bold	An element of the user interface: menu, button, field.
Settings > Stop waiting	A menu command: the menu, then the command.
:File:::PASS	Text typed exactly as shown: a command, an expected result, a file name.
Note / Caution	Information that prevents an error.

Table 1 - Typographic conventions

1.3 Terms

Term	Definition
Command row	One line of the command table: a command, its expected result and its options.
Expected result	The text that must be found for the run to be a PASS.
Result file	A text file where batch files append their time stamped PASS / FAIL lines.
Parallel command	Started together with the other parallel commands.
Single command	Started alone: "Run command(s)" waits for it before starting the next rows.
Project	A file (.pcr) holding the rows, the result file and the repeat count.
Run	One pass of "Run command(s)" over the rows; Repeat makes several.

Table 2 - Terms

2 Installation

2.1 Requirements

- Windows 10 or 11, 64 or 32 bit.
- Microsoft Visual C++ Redistributable 2015-2022 (or later) of the same bitness: without it Windows reports a missing `VCRUNTIME140.dll` or `MSVCP140.dll`.
- A folder the user can write to: the program keeps its data next to the executable.

2.2 Packages

Package	Contents	Use it when
x64 static	Process Launcher.exe, bat_examples\	normal 64-bit Windows: one file

Package	Contents	Use it when
Win32 static	the same, 32 bit	32-bit Windows
x64 / Win32 DLL	the executable and the two wxWidgets DLLs	several wxWidgets programs share the DLLs

Table 3 - Packages

There is no installer: unzip the package anywhere. Keep the DLLs (DLL packages) in the folder of the executable. The binaries are not code-signed, so Windows SmartScreen may warn at the first start (More info > Run anyway). Check a download with `certutil -hashfile "Process Launcher.exe" SHA256` against the published SHA-256.

2.3 Files and folders

Path (next to the executable)	Contents
Process Launcher data\	Created at the first start. <code>log.txt</code> (log of the sessions), <code>settings.ini</code> (result file, last project, editor text size), <code>commands.pcr</code> (the default project), the license and the trial state. If the folder cannot be created the files are written next to the executable.
<code>result.txt</code>	The default result file (see chapter 5).
<code>bat_examples\</code>	The example batch files (Appendix B).

Table 4 - Files and folders

3 The main window

From the top: the **Run command(s)** button with the **Repeat** field, the **Result file** entry, the table of commands and the list of results. The program always starts in dark mode. The status bar shows the menu help on the left and the license state on the right.

3.1 Command rows

Field	Meaning
ON / OFF	Whether Run command(s) starts the command. An OFF row is greyed out.
Run / Stop	Starts only this command, at once (it never blocks the others). While the command runs the button reads Stop: it terminates the command and everything it started (counted as FAIL).
command	Anything <code>cmd /c</code> understands: <code>bat_examples\02_ping_host.bat 192.168.1.1, ping -n 1 127.0.0.1</code> , a program with its arguments.
expected result	The text to find, the result file or the exit code to check (chapter 5).
time limit	Seconds; a command still running then is terminated with everything it started and counted as FAIL. 0 = no limit.
counters	P= nnn F= nnn: how many runs passed and failed since the program started.
Parallel / Single	Parallel commands start together; a Single command starts alone (chapter 4.3).
Hide	Reserved for showing the console of a command; not available in this version.
Busy / Ready	The command is running / idle (indicator only).

Table 5 - Fields of a command row

Note. The rows are saved in the current project when the program closes and at every Run command(s), and come back at the next start (chapter 6).

3.2 The list of results

Each run adds a line at the top: the time and `Cmd n PASS|FAIL, result is: <output>`, green for PASS, red for FAIL. A problem of the check itself (a missing file, a malformed expected result) adds a red line before it. In the list **Ctrl+A** selects every line and **Ctrl+C** copies the selected lines, tab separated (they paste into a spreadsheet as two columns); the tooltip shows the whole text of the line under the mouse.

3.3 Menus

Menu	Command	Keys	Action
File	Open project...		Opens a project file (chapter 6).
File	Save project / Save project as...	Ctrl+S	Saves the rows in the current / another project file.
File	Export results (CSV)...		Saves the results of the session as a CSV file (chapter 7).
File	Clear results		Empties the result list.
File	New batch file	Ctrl+N	Opens the batch editor with the PASS / FAIL skeleton.
File	Open batch file...	Ctrl+O	Opens a batch file in the editor.
File	Open examples folder		Opens <code>bat_examples</code> in Explorer.
File	Exit		Closes the program (see 4.7).
Settings	Enable Edit / Disable Edit	Ctrl+E / Ctrl+D	Unlocks / locks the editable fields of the rows.
Settings	Select result file...		Chooses the result file.
Settings	Open result file		Opens the result file with its default program.
Settings	Open data folder		Opens the data folder (log, settings, license).
Settings	Stop waiting	Ctrl+B	Releases the Single command (or the end of a run) that Run command(s) waits for.
Settings	Stop repeating	Ctrl+R	The current run ends, the next repetition is not started.
Info	License...	Ctrl+K	The License window (chapter 11).
Info	About...		Version, author, license, changes and system information.

Table 6 - Menus

4 Running commands

4.1 How a command runs

- The command line is run as `cmd /c "<command>"` without any visible window.
- Its current folder is the folder of the executable, so relative paths such as `bat_examples\01_minimal_pass_fail.bat` work however the program was started.
- Standard output and standard error are captured together; standard input is empty, so a command that waits for a key (pause, `set /p`) ends instead of hanging.
- Each command gets its own environment with `PCR_RESULT_FILE` (full path of the result file), `PCR_APP_DIR` (folder of the executable) and `PCR_CMD_ID` (its row number).
- The output is decoded with the OEM code page of the console, so accented letters appear correctly.

- Every command runs in its own thread: the window stays responsive. Its exit code and duration are shown in the result list; the exit code decides PASS or FAIL only with an expected result :Exit: (5.1).
- Every command runs in its own Windows job: Stop, the time limit and the closing of the program terminate the command together with every program it started.

4.2 Run command(s)

1. If a command of an earlier run is still running, the program asks whether to start the others; that command is skipped.
2. The editable fields of every row are locked.
3. The ON rows are started from the top: Parallel ones at once, Single ones as described in 4.3. A row whose expected result needs the result file is started only once the file exists (5.3).
4. Run command(s) waits until every command of the run has ended (**Settings > Stop waiting**, Ctrl+B, releases the wait), then adds a summary line: Run 1/3 ended: 4 PASS, 1 FAIL (CMD 2), green when nothing failed.
5. When the last command has ended the rows are unlocked again - unless **Settings > Disable Edit** locked them, in which case **Enable Edit** unlocks them.

The rows are saved in the current project before the run starts.

4.3 Single commands

When **Run command(s)** reaches a Single row it shows a message: the next rows wait for this command. Make sure the commands still running in parallel are not needed by it, then click OK. If any earlier row failed in this run, the program names those rows and asks whether to go on; answering No skips the Single command.

While a Single command runs the window stays usable. **Settings > Stop waiting** (Ctrl+B) stops waiting for it: the next rows start, the command goes on and its result is still recorded.

Note. Single mode needs the sequential feature of the license; without it the check box goes back to Parallel.

4.4 Repeat

The **Repeat** field next to the Run button says how many runs Run command(s) makes; each run starts when every command of the previous one has ended. 0 repeats until **Settings > Stop repeating** (Ctrl+R), which lets the current run end. After the last run a total line follows, for example Repeat ended after 100 run(s): 498 PASS, 2 FAIL. The questions of Single commands are asked in the first run only; in the later runs a previous failure is only recorded in the list. Repeat is saved in the project.

Note. Use Repeat to catch intermittent failures: the per-row counters and the CSV export (chapter 7) show which command failed and in which run.

4.5 Time limit and Stop

A row with a time limit (seconds) is terminated when it is still running after that time: the list shows CMD n terminated: time limit of 30 s expired and the run counts as FAIL. The **Stop** button of a running row does the same at once. In both cases every program the command started is terminated too.

4.6 Commands that share the result file

Commands running at the same time append to the same result file. The expected result `:File:::PASS` would also accept the PASS line of another command, so put the name of the batch file in it, as the batch files of `bat_examples` write it: `:File:::[02_ping_host] PASS`. For two rows that run the same batch file in parallel, let the batch file add its row number `%PCR_CMD_ID%` to the tag - set `"TEST_NAME=%~n0#%PCR_CMD_ID%"` - and write `{id}` in the expected result: `:File:::[02_ping_host#{id}] PASS`.

4.7 Closing the program

If commands are still running, the program asks for confirmation. Closing then **terminates the unfinished commands together with every program they started**; their results are dropped. Programs left running on purpose by commands that had already ended (for example with `start`) are not touched. Open batch editors are closed first and ask to save their changes; answering Cancel keeps the program open.

5 Expected result and result file

5.1 Forms of the expected result

Expected result	PASS when
plain text, e.g. <code>TTL=</code>	the console output of the command contains the text (case sensitive).
<code>:File:<path>::<text></code>	the file <code><path></code> contains <code><text></code> on one line. The whole file is searched. <code>%VARIABLE%</code> in the path are expanded.
<code>:File:::<text></code>	the result file contains <code><text></code> in a line written after the command started ; a PASS left by an earlier run never counts.
<code>:Exit:<codes></code>	the exit code of the command is one of the codes: a comma separated list of numbers and ranges, e.g. <code>:Exit:0</code> , <code>:Exit:0-7</code> (robocopy), <code>:Exit:0,3,10-12</code> . The output is not looked at.

Table 7 - Forms of the expected result

The first `::` after `:File:` ends the path (the colon of a drive letter stands alone). The text cannot span two lines. An expected result that starts with `:File:` but has no `::` is reported as malformed and counted as FAIL, as is a `:Exit:` without valid codes. `{id}` anywhere in the expected result is replaced by the row number.

Caution. Without `@echo off` at the top of a batch file, `cmd.exe` prints every command before running it, so the expected text is found inside the printed command: a false PASS. Keep the word PASS out of the FAIL lines and the word FAIL out of the PASS lines.

5.2 The result file

The **Result file** entry under the Run button shows its full path - by default `result.txt` in the folder of the executable. Type another path and press Enter (a relative path is relative to the executable), or click ... to choose one; the choice is kept in `settings.ini`. The file is only ever appended to, never cleared: **Settings > Open result file** opens it to read or empty it by hand.

5.3 When the result file does not exist

A command whose expected result uses the result file is started only when the file exists. Otherwise the program asks: **Create here** creates it at the path shown (with its folder), **Choose location...** chooses another place, **Cancel** leaves the command not started (a line in the list says so).

6 Projects

A project file (.pcr, INI text) holds the rows - ON / OFF, Single, command, expected result, time limit - the result file and the repeat count. The current project is named in the window title.

- The rows are saved in the current project when the program closes and when Run command(s) starts, and the project is opened again at the next start.
- Until another is chosen the project is Process Launcher data\commands.pcr.
- **File > Save project as...** saves the rows in a new file, which becomes the current project; **File > Open project...** saves the current rows, then loads another project.
- A project file can be given on the command line (chapter 8), also by associating .pcr files with the program in Windows.
- A project that cannot be read (damaged, or written by a newer version) is never overwritten: the default project is used, and a damaged default project is kept as commands.pcr.bad.

```
[Project]
Version=1
ResultFile=C:\Tests\result.txt
Repeat=1
[Row01]
Active=1
Single=0
Command=bat_examples\02_ping_host.bat 192.168.1.1
Expected=:File::[02_ping_host] PASS
Timeout=30
```

Note. A project holds at most as many rows as the window shows (11); further rows are ignored with a message.

7 Exporting the results

File > Export results (CSV)... saves every result since the start (or the last **Clear results**) as a CSV file that a spreadsheet opens in columns: the separator is the list separator of the Windows regional settings (; where the decimal sign is a comma, as in Italy), the file is UTF-8 so accents are kept.

Column	Contents
Timestamp	When the command ended.
Run	Number of the run of Run command(s); 0 = Run button of the row.
Row	Row number.
Command / Expected	What ran and what was looked for.
Result	PASS or FAIL.
Exit code	Exit code of the command, -1 if unknown.
Duration (s)	How long it ran.
Note	Why the check failed or the command was terminated.
Output	The console output.

Table 8 - Columns of the CSV file

8 Command line

```
"Process Launcher.exe" [project.pcr] [--project <file>] [--run]
                        [--repeat <n>] [--csv <file>] [--screenshots <folder>]
```

Option	Effect
project.pcr, --project <file>	Opens this project instead of the last one.
--run	Runs the rows at once without any question, exports the CSV if asked and closes.
--repeat <n>	Repeat count for this start (0 = until stopped).
--csv <file>	With --run: exports the results to this file.
--screenshots <folder>	Saves pictures of the main window, the batch editor and the About window (demonstration rows, run when licensed) and closes; used to illustrate this manual.

Table 9 - Command line options

With --run the exit code is **0** when every result is PASS, **1** when at least one is FAIL and **2** when nothing could run (no license, unreadable project, no row ON). A missing result file is created without asking. Neither the project nor the settings are written, and a project given with --project does not become the one opened at the next start. A summary line is printed to the console the program was started from.

```
start /wait "" "Process Launcher.exe" --run --project tests.pcr ^
                        --repeat 10 --csv results.csv
if errorlevel 2 echo could not run & exit /b 2
if errorlevel 1 echo at least one FAIL & exit /b 1
echo all PASS
```

Note. Process Launcher is a Windows program, not a console one: without start /wait a batch file goes on at once and the exit code is lost. In PowerShell use (Start-Process -Wait -PassThru ...).ExitCode.

9 Writing batch files

9.1 The PASS / FAIL pattern

The batch files of bat_examples report their own result: they print PASS or FAIL, end with exit code 0 or 1 and append one time stamped line to the result file. Copy 00_TEMPLATE.bat (or use File > New batch file) and put the test in the marked place:

```
if not defined PCR_RESULT_FILE set "PCR_RESULT_FILE=%~dp0result.txt"
set "RESULT_FILE=%PCR_RESULT_FILE%"
set "TEST_NAME=%~n0"

ping -n 1 127.0.0.1 >nul
if errorlevel 1 (set "MSG=ping returned an error" & goto :fail)
set "MSG=ping answered" & goto :pass

:pass
call :log PASS %MSG%
exit /b 0
:fail
call :log FAIL %MSG%
exit /b 1
:log
set "T=%time: =0%"
>>"%RESULT_FILE%" echo %date% %T:~0,8% [%TEST_NAME%] %*
echo %*
```

```
goto :eof
```

In the row: command `bat_examples\my_test.bat`, expected result `:File:::[my_test] PASS` (or simply `PASS` to look in the console output). Started by hand, the batch file writes `result.txt` next to itself.

9.2 Rules

- First line `@echo off`.
- Never pause, choice or set `/p`: there is no keyboard. To wait, use `ping -n N 127.0.0.1 >nul`.
- Append with `>>`, never `>`: the result file is shared by all commands.
- Windows (CRLF) line ends: labels and goto can fail with bare LF. The editor saves with CRLF.
- Inside a `( ... )` block `%VAR%` is expanded before the block runs: set a variable and read it in different blocks, or use labels as the examples do.
- A command that may hang needs a time limit: the time limit of its row, or one inside the batch file (see `15_timeout_guard.bat`).
- For `:Exit`: end the batch file with `exit /b <code>`.

10 The batch file editor

File > New batch file (Ctrl+N) and File > Open batch file (Ctrl+O) open the editor. On the left the text, highlighted a moment after the last key: commands and flow words in bold, comments, labels, variables and strings in their colours, `PASS` and `FAIL` standing out. On the right a button for every batch command: a click inserts it at the cursor, hovering shows its explanation (tooltip and status bar), a right click shows the details and an example; the box above the buttons filters them.

Keys	Action
Ctrl+N / Ctrl+O	New / open
Ctrl+S	Save (asks for a name the first time)
Ctrl+ + / Ctrl+ -	Larger / smaller text (A+ / A-), kept for the next time

Table 10 - Keys of the editor

10.1 Encoding of the saved file

`cmd.exe` reads batch files in the OEM code page of the console (for example 850 in Western Europe). The editor saves a file in the encoding it was read in: OEM for ordinary batch files, UTF-8 for a UTF-8 file with accented letters (a byte order mark is kept). New files are OEM. If the text holds a character the OEM code page cannot represent (such as €), the file is saved in UTF-8 and a message says so: put `chcp 65001 >nul` at the top of such a file.

Note. A batch file with accented letters saved in UTF-8 by an earlier version of the editor stays UTF-8: add `chcp 65001 >nul` to it, or retype the letters and save it as a new OEM file.

10.2 Closing

Closing the editor with unsaved changes asks whether to save them; the same question comes when the main window closes.

11 License

Running commands, Single mode and the batch editor need a license. The first start on a PC begins a **14-day trial** with every feature. Without a license the window, the About window and the License window stay available.

Feature key	Unlocks
run	the Run buttons and Run command(s)
sequential	Single commands
editor	the batch file editor

Table 11 - Features

Edition	Features
professional	all, also future ones
standard	run

Table 12 - Editions

11.1 Getting a license

1. Open **Info > License** (Ctrl+K).
 2. Click **Save Request...** and save `license-request.txt`, or copy the UID of the PC.
 3. Send it to **coga.fation@gmail.com** with the licensee name and the edition wanted.
 4. Install the `.lic` file received with **Load License...**, **Paste License**, or by dropping it on the License window.
- A license is bound to one PC and to the product ProcessLuncher; a license of another product does not unlock this one.
 - A license may end on a date and may cover the versions released up to a date ("updates until").
 - Setting the PC clock back by more than a few days suspends the trial and time-limited licenses until the date is right again.

12 Troubleshooting

Symptom	Cause and remedy
"VCRUNTIME140.dll was not found"	Install the Visual C++ Redistributable of the same bitness.
A command always passes	The batch file lacks <code>@echo off</code> , or the PASS text also appears in FAIL lines, or <code>:File:::PASS</code> accepts the line of another command: add <code>[name]</code> .
A command always fails with <code>:File:...</code>	The line is written to another file than the result file shown: the batch file must append to <code>%PCR_RESULT_FILE%</code> . The path is wrong, or the text spans two lines.
"File ... doesn't exist"	The path after <code>:File:</code> is wrong, or the command did not create it.
A command stays Busy	It waits for something (a dialog, a network). Click its Stop button, or give the row a time limit; Stop waiting (Ctrl+B) lets Run command(s) go on without it.
--run returns at once in a batch file	Use <code>start /wait "" "Process Launcher.exe" --run ...</code> (chapter 8).
Exit code 2 from --run	No license, the project could not be read or no row is ON: see the log.
"...not started: no result file"	The question about the result file was cancelled: choose the file (Settings > Select result file).

Symptom	Cause and remedy
Accented letters wrong in a batch file	It was saved in UTF-8 without chcp 65001; see 10.1.
Run buttons do nothing / ask for a license	The trial is over: see chapter 11.

Table 13 - Troubleshooting

The log of every session is `Process Launcher data\log.txt` (Settings > Open data folder); add it to a problem report sent to **coga.fation@gmail.com**.

13 Appendix A - Keyboard shortcuts

Keys	Command
Ctrl+N / Ctrl+O	New / open batch file
Ctrl+S	Save project
Ctrl+E / Ctrl+D	Enable / disable edit of the rows
Ctrl+B	Stop waiting for the Single command / the end of a run
Ctrl+R	Stop repeating
Ctrl+K	License
Ctrl+A / Ctrl+C	In the result list: select all / copy

Table 14 - Keyboard shortcuts

14 Appendix B - Example batch files

In bat_examples next to the executable. Each one names its arguments and its expected result in its first lines. They were written for Windows 10 / 11: try each by hand (double click, then look at the result file) before relying on it.

File	Checks
00_TEMPLATE.bat	skeleton to copy: put your own test in the marked place
01_minimal_pass_fail.bat	the smallest batch file Process Launcher can judge: it just reports PASS
02_ping_host.bat	a host answers to ping
03_file_exists.bat	a file exists
04_folder_exists.bat	a folder exists
05_text_in_file.bat	a text is present in a file
06_compare_files.bat	two files are identical (and how to create test files)
07_exit_code_of_program.bat	a program ends with exit code 0
08_service_running.bat	a Windows service is running
09_process_running.bat	a program is running
10_port_listening.bat	a TCP port is open (a program listens on it)
11_http_status.bat	a web address answers with HTTP status 200
12_disk_free_space.bat	a drive has at least N GB free
13_wait_for_file.bat	a file appears within a time limit
14_retry_command.bat	a command succeeds within N attempts
15_timeout_guard.bat	a command ends within a time limit (it is killed and reported FAIL otherwise)
16_registry_value.bat	a registry value contains a text
17_env_variable.bat	an environment variable is defined (and has a value)
18_copy_and_verify.bat	a copy is identical to its source
19_steps_stop_at_first_failure.bat	several checks in one batch file: the first failure stops it
20_log_with_time_demo.bat	the ways to write a time stamped line to the result file
21_arguments_and_defaults.bat	how to read arguments, give defaults and refuse a missing one
22_check_admin_rights.bat	the batch file runs with administrator rights
23_check_windows_version.bat	Windows 10 or 11 (version number 10.x)
24_dns_lookup.bat	a name resolves in the DNS
25_no_errors_in_logs.bat	no log file of a folder contains a word such as ERROR
26_file_hash.bat	a file has the expected SHA-256 hash
27_file_is_recent.bat	a file was modified in the last N minutes
28_create_and_delete_file.bat	create a file, check it, delete it, check it is gone
29_robocopy_exit_codes.bat	a folder is copied with robocopy (exit codes 0-7 are success)
30_run_another_batch.bat	run another batch file and judge its exit code

File	Checks
31_tool_installed.bat	a command line tool is installed (found in the PATH)
32_setting_in_config_file.bat	a key=value line of a config file has the wanted value

Table 15 - Example batch files

End of document PCR-UM-001, revision 1.1. Author: Fation Coga - coga.fation@gmail.com.